



# Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

## Classes: A Bond Object

WEEK 1 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# By the end of today you can

## Read a class

Tell the data from the calculations.

```
class Bond:
```

## Create a bond object

Its terms and its calculations, together.

```
qvne.current_yield()
```

## Change it safely

Mark a new price, and every number follows.

```
qvne.mark(99.25)
```

## Run a portfolio of objects

One loop. Each bond brings what it needs.

```
for bond in portfolio:
```

# So far, a bond has been three things

## Loose variables

One name for each fact.

```
ticker = 'QVNE'  
coupon = 5.25  
price = 98.50
```

## Matching lists

One list for each column.

```
coupon_list =  
    [5.25, 6.125, 4.75, 7.5]  
maturity_list =  
    [2033, 2034, 2031, 2032]
```

## A dictionary

One record, each value with a key.

```
bond = {'Ticker': 'QVNE',  
        'Coupon': 5.25}
```

All three hold the bond's data.

**None holds the calculations. Those live elsewhere.**

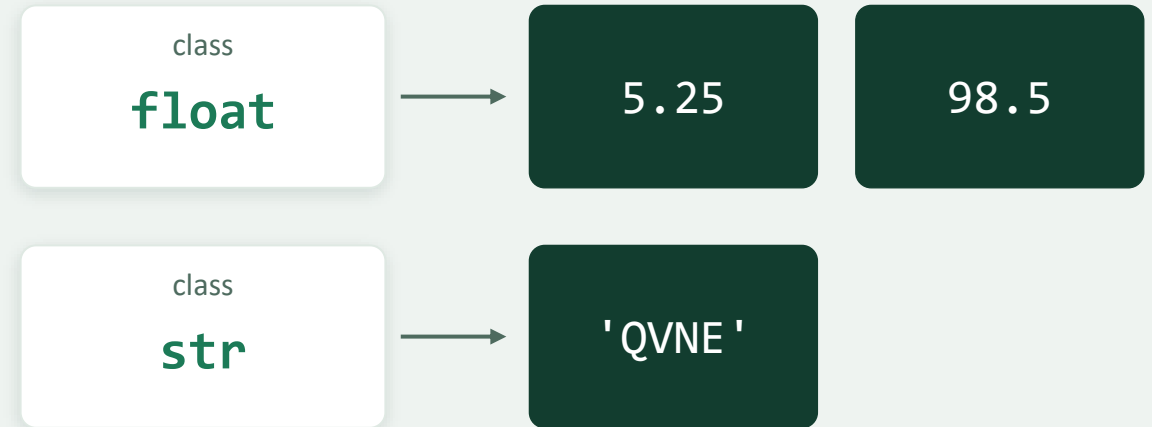
```
def current_yield(coupon, price):  
    return coupon / price * 100
```

# You have been using classes all week

Ask Python for the type

```
print(type(coupon))  
print(type(ticker))  
<class 'float'>  
<class 'str'>
```

The class is the type. Each value is an object.



A function that belongs to a class is a method

```
print(ticker.lower())  
qvne
```

## A class has two parts

The data each object holds, and the methods each object can run.

# One class, many objects

The class: a blank form

**Bond**

ticker

coupon

price

current\_yield()

It knows its own calculation.

The objects: the same form, filled in four times

**QVNE**

coupon

5.25

price

98.50

**DNMR**

coupon

6.125

price

103.25

**PLWK**

coupon

4.75

price

95.00

**HLVF**

coupon

7.50

price

101.00

# A first class, and how to use it

```
class Bond:
    def set_terms(self, ticker, coupon, price):
        self.ticker = ticker
        self.coupon = coupon
        self.price = price

    def current_yield(self):
        return self.coupon / self.price * 100
```

**self**

Stands for the object itself. You never pass it. Python supplies it.

## Attribute

Data. A dot, and no brackets.

```
print(qvne.coupon)
5.25
```

## Method

A calculation. A dot, and round brackets.

```
print(qvne.current_yield())
5.32994923857868
```

# Two ways to end up with a broken bond

## 1. Added from outside, on one bond only

```
qvne.sector = 'Energy'  
print(qvne.sector)  
Energy  
print(dnmr.sector)  
AttributeError: 'Bond' object  
has no attribute 'sector'
```

### The habit

Every object of a class has the same set of attributes. Only the values differ.

## 2. Created, and never filled in

```
plwk = Bond()  
print(plwk.coupon)  
AttributeError: 'Bond' object  
has no attribute 'coupon'
```

### The goal

Every Bond has its terms from the moment it is created.

# The anatomy of a class

```
class Bond:
    def __init__(self, ticker, coupon,
                  maturity, price, face):
        self.ticker = ticker
        self.coupon = coupon
        self.maturity = maturity
        self.price = price
        self.face = face

    def current_yield(self):
        return self.coupon / self.price * 100
```

```
# the values in the brackets are passed to __init__
qvne = Bond('QVNE', 5.25, 2033, 98.5, 1500000)
```

## class and a name

Starts the definition. The name starts with a capital.

## `__init__`

The initializer. Runs whenever an object is created.

## `self`

The object itself. First argument of every method.

## Attribute

A variable stored on the object, like `self.price`.

## Method

A function inside the class, like `current_yield()`.

*Leave a value out, and Python will not create the bond.*

# Store the facts, calculate the rest

The rule

## Attributes

The facts. Stored.

ticker  
coupon  
maturity  
price  
face

## Methods

The rest. Calculated.

current\_yield()  
annual\_income()  
market\_value()

Break the rule: store the value, then mark the price

|                | Price | Stored value | market_value() |
|----------------|-------|--------------|----------------|
| Before         | 98.5  | 1,477,500    | 1,477,500      |
| After the mark | 99.25 | 1,477,500    | 1,488,750      |

**The price changed. The stored value did not.**

No error. The object now disagrees with itself.

*A method works it out from the attributes every time, so it can never be out of date.*

# A class is a holdings sheet

Plain columns are attributes

Formula columns are methods

| Ticker | Coupon | Price  | Face      | Market value | Current yield |
|--------|--------|--------|-----------|--------------|---------------|
| QVNE   | 5.25   | 98.50  | 1,500,000 | =D2*C2/100   | =B2/C2*100    |
| DNMR   | 6.125  | 103.25 | 500,000   | =D3*C3/100   | =B3/C3*100    |
| PLWK   | 4.75   | 95.00  | 750,000   | =D4*C4/100   | =B4/C4*100    |
| HLVF   | 7.50   | 101.00 | 250,000   | =D5*C5/100   | =B5/C5*100    |

← Each row is one object

The header row and its formulas are the class.

```
for bond in portfolio:  
    total_value = total_value + bond.market_value()
```

## The fill-down is the loop

Filling a formula down the column is calling the method on every bond.

# Teaching a bond to print itself

Before: accurate, and no use to a person

```
print(qvne)
<__main__.Bond object at 0x000001ECFBDDDB230>
```

After: the way you would say it

```
print(qvne)
QVNE 5.25% of 2033
```

The method that makes the difference. It must return a string.

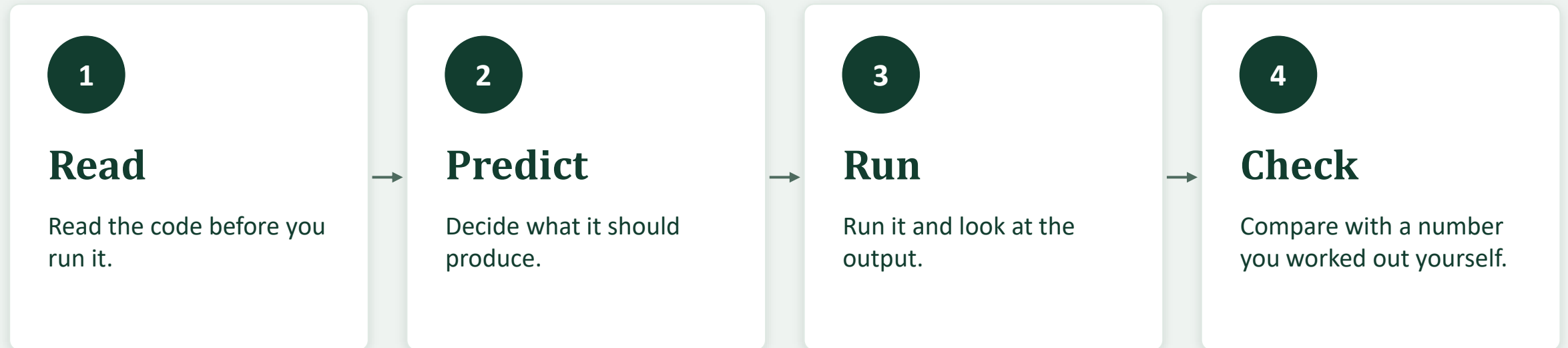
```
def __str__(self):
    return self.ticker + ' ' + str(self.coupon) + '% of ' + str(self.maturity)
```

**`__init__`** Python calls it when an object is created.

**`__str__`** Python calls it when an object is printed.

# The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.



*Code that runs is not the same as code that is right.*

# What you can do now

- Read a class: its attributes, its methods, and self
- Create a Bond object in one line with an initializer
- Store the facts as attributes and calculate the rest in methods
- Loop over a portfolio of objects for totals and filters
- Make an object print as text a person can read

**Tomorrow** Case study: organizing a small portfolio, start to finish